

Contents

MEX routines (bin)	2
NRGWilsonQS	2
fdmNRG_QS	5
fgrNRG	9
tdmNRG	12
compactQS	14
contractQS	15
diagQS	18
eigQS	18
getDimQS	19
getIdentityQS	20
getQDimQS	21
getRC	22
getSmoothSpec	22
getSmoothTDM	23
getSymStates	24
isIdentityCG	25
isIdentityQS	25
makeUniqueQS	25
mpsIsHConj	25
normQS	26
orthoQS	26
permuteQS	28
plusQS	28
setupRCStore	28
skipZerosQS	29
svdQS	29
traceQS	31
MEX routines (util)	31
filestat	31
getchar	31
kramers	32
matchIndex	32
mexworld	33
mpfr2dec	33
printfc	33
sparse2wb	34

uniquerows	34
wbeig	34
wbhash	34
wbhist	35
wbrat	36
wbsvd	37
QSpace member functions	38
QSpace/Casimir.m	38
QSpace/QSpace.m	38
QSpace/SEntropy.m	39
QSpace/SetCC.m	39
QSpace/SetOL.m	39
QSpace/acomm.m	40
QSpace/appendScalarSymmetry.m	40
QSpace/appendSingletons.m	40
QSpace/applyK1K2.m	41
QSpace/applyZ.m	41
QSpace/blkdiag.m	42
QSpace/cat.m	42
QSpace/cgr_size.m	42
QSpace/checkOM.m	42
QSpace/checkSameSpace.m	42
QSpace/checkdim.m	43
QSpace/chopd.m	43
QSpace/comm.m	43
QSpace/compact.m	43
QSpace/conj.m	43
QSpace/contract.m	44
QSpace/ctranspose.m	44
QSpace/datasize.m	44
QSpace/decomposeOp.m	44
QSpace/deconj.m	44
QSpace/diag.m	45
QSpace/dim.m	45
QSpace/disp_cgr.m	45
QSpace/disp_qdim.m	45
QSpace/display.m	45
QSpace/dsize.m	46
QSpace/eig.m	46
QSpace/eq.m	46
QSpace/expandQ.m	46
QSpace/finditag.m	47
QSpace/fixAbelian.m	47
QSpace/fixScalarOp.m	47

QSpace/full.m	47
QSpace/full2.m	47
QSpace/gdisp.m	48
QSpace/getAtensorLoc.m	48
QSpace/getData.m	48
QSpace/getEQdata.m	48
QSpace/getId.m	48
QSpace/getIdentity.m	49
QSpace/getIdentity3.m	49
QSpace/getQRange.m	49
QSpace/getQSpectra.m	49
QSpace/get_cgr_fac.m	50
QSpace/get_qdim.m	50
QSpace/getitags.m	50
QSpace/getproj.m	51
QSpace/getqdim.m	51
QSpace/getqdir.m	51
QSpace/getqfmt.m	51
QSpace/getscalar.m	52
QSpace/getsub.m	52
QSpace/getsym.m	52
QSpace/getvac.m	53
QSpace/getzdim.m	53
QSpace/gotCGS.m	54
QSpace/gotITags.m	54
QSpace/got_orthogonal_cgw.m	54
QSpace/hasQOverlap.m	54
QSpace/imag.m	54
QSpace/info.m	55
QSpace/info2.m	55
QSpace/info3.m	55
QSpace/isAbelian.m	55
QSpace/isOpConj.m	55
QSpace/isOrtho.m	55
QSpace/isQSpace.m	56
QSpace/isdiag.m	56
QSpace/isempty.m	56
QSpace/isequal.m	56
QSpace/isfield.m	57
QSpace/isop.m	57
QSpace/isreal.m	57
QSpace/isscalar.m	57
QSpace/isscalarop.m	58
QSpace/issingleop.m	58
QSpace/issingleton.m	58

QSpace/itag2iter.m	58
QSpace/itagrep.m	59
QSpace/itags2int.m	59
QSpace/itags2odir.m	59
QSpace/join.m	59
QSpace/kron.m	59
QSpace/logical.m	60
QSpace/makeIrop.m	60
QSpace/mapQ.m	60
QSpace/minus.m	60
QSpace/mrdivide.m	61
QSpace/mtimes.m	61
QSpace/norm.m	61
QSpace/norm2.m	61
QSpace/not.m	61
QSpace/numsym.m	61
QSpace/olap.m	61
QSpace/oplus.m	62
QSpace/permute.m	62
QSpace/plotEdata.m	62
QSpace/plotQSpectra.m	62
QSpace/plot_op.m	63
QSpace/plus.m	63
QSpace/randomize.m	63
QSpace/rank.m	64
QSpace/real.m	64
QSpace/rgbQSpectra.m	64
QSpace/rmitags.m	64
QSpace/rmtrace.m	64
QSpace/sameas.m	65
QSpace/setitags.m	65
QSpace/showdeg.m	66
QSpace/skipprec.m	66
QSpace/skipzeros.m	66
QSpace/sort.m	66
QSpace/split_ops.m	67
QSpace/sqrt.m	67
QSpace/squeeze.m	67
QSpace/subsasgn.m	68
QSpace/subsref.m	68
QSpace/sum.m	68
QSpace/symperm.m	68
QSpace/symrank.m	68
QSpace/tensor.m	69
QSpace/times.m	69

QSpace/toStr.m	69
QSpace/trace.m	69
QSpace/transpose.m	69
QSpace/tstfun.m	69
QSpace/uminus.m	70
QSpace/untag.m	70
QSpace/uplus.m	70

MEX routines (bin)

>> NRGWilsonQS -h

Usage: [[NRG_DATA,] info] = ...

NRGWilsonQS(H0, A0, Lambda, ff, FC, Z, [, gg, FL, OPTS])

NRG run using (non-)Abelian symmetries for a Wilson chain of length L based on the standard iterative diagonalization prescription.

The routine performs a quick check whether any of the inputs is complex. This includes the QSpaces H0, A0, FC, FL (if present), or the hopping amplitudes ff. If so, NRGWilsonQS runs in full complex arithmetic, thus returning complex AK and AD. The subsequent `fdmNRG_QS` also can deal with either case. All routines use double precision throughout.

- H0 impurity / starting Hamiltonian (rank-2 QSpace)
H0 will be diagonalized upon initialization hence does not have to be diagonal on input. The resulting basis transformation is multiplied onto A0.
- A0 basis used for H0 in LRs order (rank-3 QSpace)
where the local space $s(\text{igma})$ must already have the (equivalent) space of a Wilson site (note that A0 is used to obtain the matrix elements of FC at the first iteration.
- Lambda used to rescale H_k by $\sqrt{\text{Lambda}}$; therefore this should be consistent with the Lambda used to obtain ff and gg below
- ff strength of couplings along Wilson chain as they appear in the physical Hamiltonian, i.e. they must not be rescaled yet, as this is done automatically based on Lambda! if ff is a vector of length L-1, the coupling applies for all nearest-neighbor hoppings via the FC operators in a diagonal fashion. ^1)
If ff is a matrix of dimension (L-1) x Nf, the diagonal hopping matrix elements can differ across the FC operators. If ff is a matrix of dimension (L-1) x (Nf*Nf), this also permits to encode cross-couplings $FC(i)'*FC(j)$ in between nearest-neighbor shells.
- NB! The length L of the Wilson chain is determined by the length of ff, i.e. $L=\text{length}(ff)+1$ by including A0.
- FC set of Nf annihilation operators used for coupling (QSpace)
e.g. in case of all-abelian symmetries (default) this becomes

$H = F1' * ZF2 + F1 * (ZF2)' + h.conj.$ (zflag=1, see below)
 with $F1=F2=FC$ such that this operator always appears as
 a bilinear together with its hermitian conjugate. ^{^2)}

Z (diagonal) operator required for the calculation of the
 fermionic hopping matrix elements (QSpace). It is defined by
 fermionic signs $(-1)^{ns}$ required in the context as described
 for FC above, where ns is the number of fermionic particles
 for the local states s(igma).

Options

zflag variation of how to apply the fermionic signs Z on the
 hopping terms in the Hamiltonian determined by the operators
 FC (default: 1 e.g. as described for FC above in the context
 of all-abelian quantum numbers). Valid settings:

- 0: ignore Z altogether
- 1: $H = F1' * (Z * F2) + F1 * (Z * F2)' + \text{hermitian conjugate}$ (default)
- 2: intended for particle-hole-symmetric cases only:
 $H = (ZF1)' * (ZF2)$ at every second iteration only, WITHOUT
 adding a further hermitian conjugate (already included!)
 starting with NO Z-operators applied to A0.local (i.e.
 same as for zflag==1), therefore starting with $F1' * F1$
 at first iteration (i.e. using F1 to update operators,
 and again F1 when building enlarged Hamiltonian.
 For every other iteration, $H = (Z * F1)' * (Z * F2)$ is applied
 using Z with BOTH operators, consistent with particle-
 hole symmetry!
- 3: same as zflag=2, but Z is applied at every other iteration
 already starting with the very first iteration.

FL set of Nl local operators along the chain (QSpaces [Nl])

gg strength of local operators FL along the chain similar to ff
 i.e. not rescaled! (double, (L-1) x Nl, not including H0) ^{^1)}
 Note that local operators can always be expressed as
 Hermitian operators, with real amplitudes gg. Hence gg is
 always expected to be real.

^{^1)} NB! if the couplings ff (or also gg) are the same for all FC and FL,
 they may be represented by a single vector only.

^{^2)} FC and FL are considered the same for every iteration throughout
 the Wilson chain, yet their strength is modified by ff and gg,
 respectively.

Further options / flags

'Nkeep',.. max number of states to keep at every NRG iteration (512)
'NKEEP',.. vector that specifies Nkeep individually for first few sites

'Etrunc', use given energy threshold rather than Nkeep to truncate
 unless Nkeep is reached (≤ 0 uses Nkeep only; default: 0).
'ETRUNC', vector that specifies Etrunc individually for first few sites
'ET1' Etrunc for 1st with truncation (default: $1.2 \times \text{Etrunc}$)
 if $\text{ET1} < 0$, then $\text{ET1} = \text{Etrunc}$ is used.

'FNfac',.. global factor on energy scale, i.e.
 $\text{EScale}(n) = \text{Lambda}^{(-n/2)} * 0.5 * (\text{Lambda} + 1) * \text{FNfac}$.
 (by default, FNfac is chosen such that $\text{ff}(N-1)/\text{EScale}(N)=1$;
 see returned info.EScale below)

'Estop',.. stop dynamically once stable fixed point is reached, i.e.
 energies $\text{EK}(1:\text{Nkept}/4)$ for iterations k and $k+2$ are the same
 to within given value for Estop (default: 0, i.e. not used).
'-Estop' same as 'Estop', $1\text{E-}4$

'NEE',.. number of states to store for energy flow diagram ($\text{Nkeep} * \text{Ns}$)
'fout',.. store data for every site into output file <fout>_##.mat
 if specified, the first RETURN argument is skipped (./NRG/NRG).
'deps',.. restore degeneracy within eps ($1\text{E-}12$)
'db',.. require minimum distance towards discarded energies (-1)
 ($\text{db} < 0$ searches for maximum distance in vicinity of Nkeep)
'dmax',.. maximum number of extra states to include for db ($\text{Nkeep}/10$)
 (mostly a safeguard in case db has been set too large).
'-IdF' build local Id from F-ops, fully ignoring A0 (NB! by default,
 sigma-space of A0 is considered a proper Wilson bath site!)
'ionly' info only (incl. Eflow) i.e. do not store iterative NRG data
'-v' verbose mode

Output

NRG_DATA MPS data (only if 'fout' was not specified; otherwise this
 data is stored into specified file structure):

 .A[KD] states $K(\text{ept})$ / $D(\text{iscarded})$ at current NRG iteration
 according to NRG truncation scheme (rank-3 QSpaces in LRs order)
 .H[KD] eigenenergies corresponding to the eigenbasis of AK/AD ;
 stored in diagonal format.

info info structure containing the following fields:

.istr info string
.usage (struct) info such as time stamp, etc.
.HK rescaled energy data for energy flow diagram (QSpace vector)
.EE collected eig(HK) data (purely numerical array)
.E0 subtracted ground state energy at energy scale of iteration
.EK [largest kept, lowest and largest discarded] energy for each iteration
.EScale NRG energy scale used along the Wilson chain (see FNfac above)
.Lambda Lambda used in the calculation (same as input)
.ops (struct) operator specific data for Wilson chain (FC,Z,ff,gg,etc.)
.paras (struct) technical NRG specific internal parameters

Alternative usage: NRGWilsonQS('MATfilename')
NRGWilsonQS('setup.mat' [, 'fout_tag'])

This allows to read all input parameters from given setup.mat file (MatLab binary) which allows pure standalone run of NRG without requiring MatLab (except for the API for binary *.mat I/O).

This way all variables and options MUST be defined by their names shown above; moreover, this usage must include a specification for fout_tag as all output data will be written there. fout_tag is either specified as 2nd argument above, else it should be defined in terms of the variable 'fout' in the parameter file setup.mat (default for fout_tag: './NRG/NRG').

AW (C) Apr 2006 ; May 2010 ; Nov 2014

>> **fdmNRG_QS** -h

Usage: [omega, A0 [,Info]] = ...
 fdmNRG(NRG_DATA [,Inrg], B, C, Z0, [,opts])

This routine calculates arbitrary correlation function (CF) in the full density matrix Numerical Renormalization Group (fdm-NRG) framework at arbitrary temperature T for arbitrary abelian or non-abelian symmetries.

$$A(w) = \langle B || C' \rangle_w = \text{FFT}(\langle \{B(t), C'\} \rangle). \quad (' \text{equiv } ^\dagger)$$

If B is specified as empty [], it will be automatically assumed equal to C, i.e. B=C as is the case for spectral functions; anti-commutator (default) or commutator will be chosen according

to cflags below (see also zflags for Fermionic signs).

This routine uses data from a previous standard NRG run with the result written as a matrix product state in QSpace format. It must contain the state space kept (AK) as well as the one thrown / discarded (AD) at every NRG iteration.

See also: NRGWilsonQS

Input:

NRG_DATA	structure containing data of previous NRG run; if this contains path/file, then the data will be read from the files path/file_##.mat for every site (see NRGWilsonQS)
.AK	MPS state of k(ept) space (regular NRG, QSpace)
.AD	MPS state of t(runcated) NRG space (QSpace)
.HK	effective NRG eigenspectra (kept, QSpace)
.HD	effective NRG eigenspectra (truncated, QSpace)
Inrg	NRG info structure (required if NRG_DATA contains structure; otherwise <NRG_DATA>_info.mat will be accessed. required elements within Inrg structure / data:
.Lambda	NRG discretization parameter for the conduction band. Lambda is needed to undo the energy scaling performed in obtaining the NRG data at the first place.
.EScale	energy scale fore each Wilson shell
.E0	(relative) ground state energy offsets in the same energy scales as H[KD]; this is needed for evaluating the correct weights of the density matrix
B,C	operators acting on the impurity used to calculate the correlation between them as in $A(w)=FFT(\langle\{B(t),C'\}\rangle)$.

If the operators B or C are specified by name, they will be read from workspace, and stored in NRG_DATA together with the other NRG MPS data (the latter is disabled by the flag 'nostore')

The operators in B and C are paired up for correlation functions. They are applied in the order of priority (whichever fits first): L, then s, then R onto A0

as in <NRGDATA>_00 with index order LRs where L = left,
R = right = combined, s = local state space sigma).

Z0 Z0 operator for the space described by B and C
 required for fermionic setup where must represent
 $Z0=(-1)^{(\text{number of particles})}$; Z0 is directly
 contracted onto the B and C where 'zflags' is set.

Options / Flags (default method: fDM)

'fDM' full-density-matrix approach to CFs (default)
'NRho',k, build density matrix from single iteration k
 (for testing purposes, only)

'T',... temperature at which correlation function is calculated;
 default: about an order of magnitude larger than the
 energy scale at the end of the Wilson chain

'vflag',... more detailed output with increasing value (default: 1)
'partial' store G1 and G2 separatly (e.g. for detailed balance
 or the calculation of expectation values)
'nostore' do not store operator elements of B or C with NRG_DATA
'locRho' if Rho needs to be calculated it is stored in RAM only
 and will be deleted after the program is finished
'noRHO' do not calculate Rho, eg. calculate partition function
 only (relevant with empty ops B and C only)
'rhoNorm',.. allows to specify the weights rhoNorm explicetly
 as input (intended for testing purposes only)

'zflags' on which operators in <B||C> to apply fermionic signs Z0
 ex. 'zflags', [1 0 ...] would apply Z0 to the first
 but not to the second operator (default: all 0 if Z0
 is empty, 1 otherwise).

'nlog',... number of bins per decade for log. discretization (256)
'emin',... minimum energy for omega binning (TN/1000) ^1)
'emax',... maximum energy for omega binning (10.) ^1)

^1) NB! first (last) bin contains all data below (above) emin (emax)

Output

om omega corresponding to the energy binning [-emax,+emax]
A0 raw data for spectral function (i.e. discrete in om)

Info info structure: specific fields are

.ver version / mode of code

.T actual temperature used

.rho FDM weight distribution for given temperature

.RHO reduced thermal density matrix for A0 (R-basis).

.Om $\Omega(T) = -T \log(Z)$, (grand)canonical potential.

.A4 individual spectral data for each of the two
contributions to the (anti)commutator (hence A4 has
twice as many columns as A0 above).

.a4 (1st row) integrated spectral integrals over the two
individual contributions to the (anti)commutator of
given correlator (2nd row may be ignored; it refers
to check w.r.t. detailed balance).

.reA0 contains $\text{Re}(G(\omega \rightarrow 0))$ for the calculated correlation
functions. Importantly for finite temperatures, the
correlators with $\text{cflags} \neq 0$, i.e. susceptibilities,
acquire in addition to the Kramers Kronig transformation
(principal value integral!) also a Matsubara correction
for $E_a = E_b$, which would be skipped in the principal
value integral; this correction [see Hanl and AW, PRB 2014]
is added to the first term in the commutator.

.symfac IROP factors for given spectral functions.

ALTERNATIVE USAGE

```
fdmNRG('--mat', 'nrgdata', 'setup.mat' [, 'B'], 'C')
```

B and C are the same as above (B=C if B is not specified).
This usage allows to read all input parameters from given MAT files
(MatLab binaries); all variables and options MUST be defined in
<setup.mat> by their names shown above; e.g. the operators B and
C are specified by their names as defined in <setup.mat>.

The output will be written / updated to the file set
'<nrgdata>_##.mat' and '<nrgdata>_info.mat'.

Output will be appended to setup.mat.

See also NRGWilsonQS.

AW (C) May 2006 ; May 2010 ; Nov 2014

>> fgrNRG -h

Usage: [[[om,] a0], Ifgr] = fgrNRG(NRG0, NRG2, Lambda, B [,C], [,opts])

full density matrix (FDM) Numerical Renormalization Group (NRG)
calculation for Fermi Golden Rule (FGR) rates, with extension
to resonant inelastic x-ray scattering (RIXS).

FGR mode (default)

This computes the FGR spectral data (om,a0) within the fdm-NRG
framework at arbitrary temperature (default $T=T_N$ with N the
length of the Wilson chain). When Fourier transformed, it corresponds
to the time-dependent correlation function [see Ifgr.ltdm.at]

$$a(t) = \langle B(t)C' \rangle = \text{tr}[\rho_0 * B(t) C']$$

Here B and C are local operators acting at or in close vicinity
of the impurity. They are assumed to switch back and forth
between two disconnected sectors of Hilbert space (e.g. by the
presence of an (otherwise unaccounted for) valence core hole.
The time evolution $C(t) = \exp(iH_0 t) C \exp(-iH t)$ therefore becomes
'mixed' with 'initial' Hamiltonian H_0 and final Hamiltonian,
iteratively diagonalized for both, NRG0 and NRG2, respectively.
The frequency corresponds to $\omega = E_{\text{final}} - E_{\text{initial}}$.

If C is not specified or empty, it is automatically assumed
equal to B. This is the typical situation in FGR.
The initial state is determined by the full density matrix (FDM)
 ρ_0 of NRG0 at given temperature (default $T \sim T_N \sim 0^+ K$).
The dynamics occurs within NRG2.

Input

NRG*	(string pointer to) data of previous NRG runs; if this contains <path/file>, then the data will be read from the files <path/file>_##.mat files as generated by NRGWilsonQS.
Lambda	NRG discretization parameter used in NRG*. While EScale and E0 will be read from NRG0_info, Lambda is needed for simple estimates of energy scales.
B,C	local operators, e.g. acting on the impurity. *) If operators B or C are specified by name, they will be

read from workspace and stored in NRG2_info
(this is disabled by 'nostore').

NB! Note that exchanging NRG0 <> NRG2 and B<>C corresponds
to changing from emission to absorption spectra or vice versa.

Options / Flags

'Eoffset',.. additional energy offset which affects the omega
binning (om-binning) only. Eoffset acts *on top* of
EO_final-EO_initial, which ensures that by default,
(Eoffset=0) omega=0 corresponds to minimal excitation
energy for a transition from groundstate in NRG0
to groundstate in NRG2 (with NRG2 considered (much)
higher in energy in x-ray absorption).

'T',... temperature at which correlation function is calculated (0.)
'NRho', k builds density matrix from iteration k only (cf. 'DM-NRG')

'emin',... min. energy for omega binning (TN/1000) **)
'emax',... max. energy for omega binning (10.)
'nlog',... number of bins per decade for om-binning (256)

'nostore' do not store operator matrix elements of C[12] with NRG data
'locRho' if Rho needs to be calculated it is stored in RAM only
and will be deleted after the program is finished

'sigma',... applies broadening in frequency domain (log.gauss)
NB! either alpha or sigma can be specified but not both.
default sigma=0.3 (with alpha ignored)

'alpha',... exponential decay of energies $\exp(-\alpha \text{ dE})$ [0.]
this applies alpha in time domain (F. Anders 2005)

'partial' return partial data from every NRG iteration
'raw' raw data only, i.e. no smoothing of data
'RAW' calculates time dependent data from exact frequency data
i.e. without binning of energies on log. scale
'disp' more detailed output (0)

'Z0',... Z0 operator for first site (impurity) required for
fermionic setup where $Z0=(-1)^{(\text{number of particles})}$

'zflags',... on which operators in B.*C to apply fermionic signs Z0
ex. 'zflags', [1 0 ...] would apply Z0 to the first
but not to the second operator (default: all 1s,
as typically the case for absorption/emission setups.

RIXS mode

With the flag '--rixs', this routine computes discrete, i.e., binned, RIXS (resonant inelastic x-ray scattering) energy loss spectra for given incoming photon energy E_{inc} . This uses the Kramers-Heisenberg higher-order generalization of Fermi golden rule.

Note on semantics: within RIXS, $i(\text{nitial})$ and $f(\text{inal})$ refer to the **same** Hamiltonian, whereas it is the intermediate Hamiltonian takes the role of the former $H_2 = H_{\text{final}}$. With

$$V_{\text{if}} := \langle i | B \frac{1}{[E_{\text{inc}} + E_i - H_2]} C | f \rangle$$

the RIXS spectral data would correspond, when Fourier transformed into time-domain, to

$$R(t) = \langle V(t) V' \rangle = \text{tr}[\rho_0 * V(t) V']$$

thus recovering the same final form as for absorption/emission spectra above. The RIXS mode requires the additional parameter

' E_{inc} ',... incoming photon energy $E_{\text{inc}} = \text{complex}(\omega_{\text{in}}, \gamma)$; this must already also include the broadening γ related to finite life-time of the core-hole due to other unaccounted for relaxation processes such as Auger etc.

The returned spectra correspond to the RIXS intensity, i.e., spectral intensity a_0 vs. energy loss ω .

Output

ω , a_0	binned spectral data (according to $\epsilon_{\text{min}}, \epsilon_{\text{max}}, n_{\text{log}}$)
Ifgr	info structure

Notes / comments

- *) Alternatively, if B and C have composite structure that act on multiple sites/indices simultaneously, the operators B and C can also be specified as cell arrays of QSpace vectors. A given QSpace vector then, e.g., $B\{i\}$, is interpreted as the sequence of operators that act like a tensor product on indices $L_{00}, s_{00}, s_{01}, \dots$ where L_{00} and s_{00} are the 'left' and local ('sigma') indices of A_0 , respectively, and s_{01} , etc. the local indices

of A01, etc. In this usage, Z0 must be already properly included in B and C upon input, i.e. Z0 and zflags cannot be used.

**) The bin a0(end) collects all data with $\omega \geq \omega(\text{end}) = \omega_{\text{max}}$. For ω_{min} , the smallest bin a0(1) (given the logarithmic grid!) contains all data where ω is exactly zero (e.g. for diagonal contributions in Lehmann representation). The next smallest bin a0(2) then contains all data for $0 < \omega \leq \omega(2)$.

Alternative usage:

```
fgrNRG('nrgdata0','nrgdata2','setup.mat','B','C')
```

This reads all input parameters and options from given setup.mat file. Hence the operators B and C must be specified by their name. Output will be written / updated to NRG0 and NRG2, with results also appended to setup.mat.

See also NRGWilsonQS, fdmNRG_QS, tdmNRG

AW (C) 2008 ; RIXS 2019

>> **tdmNRG** -h

Usage: [cc [,Info]] = ...

```
tdmNRG(NRG0, NRG2, Lambda, C1 [,C2], tt, [,opts])
```

time-dependent numerical renormalization group (NRG)
using the full thermal density matrix of the initial system
(FDM), rho_0, as starting configuration at time t=0.

This routine calculates the time evolution of local operators C1(t)*C2 in fDM-NRG framework at given temperature T.

$$cc(t) = \langle C1(t)C2' \rangle_0 = \text{trace}(\rho_0 * C1(t) * C2')$$

With option -fgr the initial density matrix is modified by the operator C2 (e.g. creation / deletion of a particle as in absorption / emission processes) with the resulting modified time evolution being

$$cc(t) = \text{trace}((C2' \rho_0 C2) * C1(t))$$

Hence the operator C2 appears twice. Note, however, that the projected density matrix C2' rho_0 C2 is no longer

normalized.

This routine uses data from two previous NRG runs with the result written as matrix product state in QSpace format. It must contain the state space kept (AK) as well as the one discarded (AD) at every NRG iteration.

The initial state is determined by the density matrix ρ_0 of NRG0 at given temperature T . The time evolution is determined within NRG2.

See also: NRGWilsonQS, dmNRG, fgrNRG

Input:

NRG[02] specifies previously calculated NRG data; if this contains path/file, then the data will be read from the files path/file_##.mat for every site (see NRGWilsonQS)

.AK MPS state of k (ept) space (regular NRG, QSpace)
.AD MPS state of t (runcated) NRG space (QSpace)
.HK effective NRG eigenspectra (kept, QSpace)
.HD effective NRG eigenspectra (truncated, QSpace)

Lambda NRG discretization parameter for the conduction band. Lambda is needed to undo the energy scaling performed in obtaining the NRG data at the first place.

C[12] local operators, e.g. acting on the impurity

If C2 is not specified or specified as empty [], it will be automatically assumed equal to the identity operator.

NB! If operators C[12] are specified by name, they will be read from workspace and stored in NRG_DATA together with the other NRG data (can be disabled by flag 'nostore').

Options / Flags (default method: tDM)

'alpha',... exponential decay of energies $\exp(-\alpha dE)$ [0.]
this applies alpha in time domain (F. Anders 2005)
'sigma',... applies broadening in frequency domain (log. gauss)
NB! either alpha or sigma can be specified but not both.
default: sigma=0.3 with alpha ignored.

'NRho', k builds density matrix from iteration k only.
 'T',... temperature at which correlation function is calculated.
 If not specified, the temperature T is given by the
 energy scale towards the end of the Wilson chain.

 '-fgr' run in FGR modus applying C2 twice (see above).

 'nlog',... number of bins per decade for logarithmic discretization (256)
 'emin',... minimum energy for omega binning (TN/1000)
 'emax',... maximum energy for omega binning (10.)

 'nostore' do not store operator elements of C[12] with NRG_DATA
 'locRho' if Rho needs to be calculated it is stored in RAM only
 and will be deleted after the program is finished
 'partial' return partial data from every NRG iteration
 'raw' raw data only, i.e. no smoothing of data
 'RAW' calculates time dependent data from exact frequency data
 i.e. without binning of energies on log. scale
 'disp' more detaild output (0)

 Z0 Z0 operator for first site (impurity) required for
 fermionic setup where $Z0=(-1)^{(\text{number of particles})}$
 'zflags' on which operators C[12] to apply fermionic signs Z0
 ex. 'zflags', [1 0 ...] would apply Z0 to the first
 but not to the second operator (default: none).

NB! The first (last) bin contains all data below (above) emin (emax)

Output

cc expectation value at times specified in tt
 Info info structure

See also NRGWilsonQS, fdmNRG_QS, fgrNRG
 AWb (C) Jan 2007

>> compactQS -h

Usage: S=compactQS([opts], 'qtype', Q1, Q2, Q, A [,perm]);

reduce matrix elements of the rank-3 IROP.
 Here A is still specified in full (dense) tensor format
 with matrix elements given by

$A(i1,i2,io) := \langle i1|A(io)|i2\rangle$

NB! this assumes the index (1,2,op) specified on the lhs!
 The symmetry types are specified by qtype. Moreover,
 all states i1 and i2 must be already cast into symmetry
 eigenstate grouped into multiplets (see getSymStates.cc)
 with q- and z-labels both present and interleaved in
 each input state label sets Q*.

The state spaces are combined in the order (Q2+Q =>Q1),
 thus using the Clebsch Gordan coefficients (Q1,Q|Q2).
 This is a natural order in that, Q=-1 annihilates a
 particle on a |ket> state (the annihilation operator
 on a ket state increases charge, i.e. Q would be +1,
 in case Q1 and Q had been combined into Q!).
 This therefore represents a natural index order.

Options

perm permutation to be applied to {Q1,Q2,Q} and A
 prior to combining first two indices into third.

'-h' display this usage and exit

(C) AW : Apr 2010 ; Jul 2011 ; Oct 2014

>> **contractQS** -h

Usage: **contractQS**(A,...B,...)

performs pairwise contraction of tensors
 e.g. A and B in QSpace format while also taking care of the
 underlying Clebsch Gordan coefficient spaces (if present).

Each QSpace can be used as is, or as its 'conjugate' where the
 conjugate of a QSpace A, i.e. conj(A) is defined as the QSpace

- a) with all 'arrows' reversed
- b) keeping the SAME qlabels [for reversing individual
 arrows, see getIdentitQS(..,'-0')]
- c) and complex conjugation of all data{*}
 if applicable, i.e. a given QSpace is complex

Note that because of (a), the specification of conjugation flags

('conj-flags') is also important for QSpaces with all-real matrix elements.

Usage #1: `S=contractQS(A, ica, B, icb [, perm, OPTS ])`;

Plain contraction of a single pair of QSpaces, A and B, with respect to given explicitly specified sets of contraction indices ica and icb, which can be specified

- numerically (e.g. `[1 2]`),
- or as strings (e.g. `'1,2'`, or `'1 2'`)
- or as compact strings (e.g. `'1,2'`, or `'12'`)

The last 'compact format' is only possible / unique, of course, if the tensors A and B do not have more than 9 legs (which basically never occurs), such that the contraction indices remain in the single digits (this can be further relaxed, though, by extending the digital range to letter, i.e. using a-z after 0-9).

The recommended way to specify conj-flag with usage #1 is together with the contraction indices in string notation! For example,

```
contractQS(A,[1 3],B,[1 3],'conjA') is equivalent to
contractQS(A,'1,3;*',B,[1 3])        is equivalent to
contractQS(A,'13*',B,'13')
```

(Deprecated) options specific to usage #1:

```
'conjA' use complex conjugate of A for contraction
'conjB' use complex conjugate of B for contraction
```

Usage #2: `S=contractQS({A,{B,C}},... [, perm, OPTS ])`;

Generalized 'cell-contraction' of tensors: when encountering a cell, the content of that cell is contracted first, before using its results. This allows the specification of an entire pattern of pairwise contractions based on nested structures where the lowest-level contractions are performed first.

Cell contractions are furthermore based on QSpace 'itags' i.e. string labels for indices with up to 8chars, and which are specified in `QSpace.info.itags`. This offers automated contraction ('auto-contraction') of pairs of tensors solely

based on matching itags! Usage #2 therefore does not (also) allow explicit specification of contraction indices as in usage #1. Therefore itags (plus conj-flags) must be unique.

Itags must always also contain individual conjugate flags (this represent the bare minimum that must be specified with each QSpace in v3): the conjugate flags on individual indices (legs) of a tensor determine whether that index (leg) is in- or out-going, with the convention that

all out-going indices have a trailing * in their itags!

For example, an A-tensor with L(ef), R(ight), s(=local) indices may have itags `A.info.itags={'L','R*','s'}` assuming (L,R,s) index order.

In usage #2, for every operator additional optional strings can be specified, appearing right after the affected tensor e.g. QSpace A:

A,'!ij' do not contract indices specified by ij (in compact format) despite they share common matching itags.
A,'*' apply overall (complex) conjugation on given input tensor A (see early comments above)
A,'!ij*' both of the above (with * always trailing).

X,'-op:<tag>[:<opl=op>]' specify itags for given (e.g. local) operator.

The last option considers X an operator, and hence assumes operator itags '`<tag>;<tag>*[:opl*]`' for QSpace X; the default operator label is 'op', but may be changed by specifying a trailing '`<opl>`' as indicated above. As a safeguard, this issues a warning, if existing itags are overwritten. This is relevant e.g. for local operators that are applied to a very specific site with associated itag.

[11/24/2018] the specified <tag> may now also represent a regular expression (regex), recognized by non-alphabetic special characters, while ignoring conj-flags (without special characters, the specified <tag> is taken as is!). This usage then searches for a matching itag in the paired up QSpace (cell) in the contraction.

Ex. Consider A some QSpace with a single local index that starts with 's', e.g., like 's010' for site 10;

then `contract(A,Xloc,'-op:~s')` will autocontract the local operator `Xloc` to the correct local index in `A` (here the regex `'~s'` indicates 'starts with s'); An alternative operator `itag` may still be specified by adding a trailing `':opl'` as indicated earlier.

An adaptation of usage #2 also be used for a plain sequential contractions where

```
S=contractQS(A,B,C,... [, perm, OPTS ]); is equivalent to
S=contractQS({A,{B, {C,...}}}, [,perm,OPTS ]),
i.e. the sequential contractions are started from the end
onwards to the beginning of the set.
```

The remaining trailing `OPTS` are

`perm` permutation to be applied to the final object;
NB! [06/02/2019] this permutation can be shorter than the rank of the resulting `QSpace`; in this case it only affects the leading range of indices.

`'-v'` verbose mode that shows level of cell contraction together with actual contractions performed.

Mixed usage of #2 and #1 is not possible.

Note that mex files do not allow to return class objects, hence `S` is returned as a `QSpace` structure. To get a class object, use `QSpace(contractQS(...))`, or equivalently, if `contract()` is properly defined as a wrapper routine within matlab's `Class/@QSpace` (see `MPS Pack`), having a `QSpace` input `A`, this may be shortened to `contract(A,...)`.

AW (C) May10 ; Aug12 ; Dec14 ; Sep16

```
>> diagQS -h
```

Usage: `dd=diagQS(A)`

Get diagonal data of given `QSpace` (for rank-2 `QSpaces` only).

Wb,Aug16,16

```
>> eigQS -h
```

Usage: `[E [,I]]= eigQS(H [,opts])`

Obtain eigenspectrum and eigenbasis of input 'Hamiltonian' H in QSpace representation. The eigendecomposition is split into kept (K) and discarded (T) state space, as governed by the truncation parameters Nkeep and Etrunc (for actual Hamiltonians) or Rtrunc (for density matrices $R:=H$).

Options

'Nkeep',... max. number of states to keep (default: -1 = keep all)
'Etrunc',... keep states according to energy truncation criteria with $E \leq E_0 + E_{\text{trunc}}$ (default: ≤ 0 = ignore; note that Etrunc is interpreted relative to lowest eigenvalue $E_0 = \min[\text{eig}(H)]$).
'Rtrunc',... keep states according to truncation criteria of reduced density matrix with $\text{eig}(R) > R_{\text{trunc}}$ with $R:=H$ (this is alternative to Etrunc and thus ignored by default; note, furthermore, that here Rtrunc is interpreted absolute, i.e. as is).
'deps',... allow to keep (close to) degenerate spaces together (default: 0, i.e. sharp truncation e.g. with Nkeep)

Output

E contains the full sorted eigenspectrum (1st column), and in the presence of non-abelian symmetries, the overall multiplet degeneracy as a 2nd column.

The output/info structure I contains the fields

.AK, .AD for kept and discarded state space
.EK, .ED, for kept and discarded eigen spectrum (diag)
.DB block dimension of input Hamiltonian H in terms of multiplets (1st col) and states (last col).
.NK actual number of kept multiplets / states.
.Etrunc or
.Rtrunc actual truncation threshold used

AWb (C) Sep 2006 ; Mar 2013 ; Aug 2015

>> getDimQS -h

Usage: [D[,D2]] = getDimQS(A)

get full dimension of given QSpace object.

In case of non-abelian symmetries, usage #2 returns number of multiplets as D, and the full state space dimension as 2nd argument D2.

NB! if no second output argument (D2) is requested, this routine returns D if D==D2 (e.g. which is always the case of abelian symmetries) but [D;D2] if D!=D2.

(C) AW : May 2006 ; Oct 2014

>> **getIdentityQS** -h

Usage 1: A=getIdentityQS(A [,i1, perm,'-0']);

get plain identity operator from given QSpace.

'-0' this generates the 1J symbol, i.e. an identity operator with all indices inward. This can be used to "revert arrows", and subsequently also to avoid dragging along a scalar singleton index if the total symmetry of a given wave function has q=0.

Usage 2: A=getIdentityQS(A [,i1 [,at]], B [,i2 [,bt] itag, perm]);

get tensor product space of input spaces defined by A and B. If i[12] is not specified, rank-2 objects are assumed taking (the first) two indices.

Default index order of the output space is (1,2,12*) with the itags of A and B inherited if present. An <itag> name for the combined output space (dim3) may be specified (conj flag if present, will be ignored).

[06/01/2019] In addition, for an input A-tensor with LRs index order convention, its itags may be inherited to C by specifying xor <bt> (see usage above) in the format '-A[:...]' where '...' indicates extra characters to be catenated at the end of the itag for the fused index.

For example, consider an A-tensor with itags {K01,K02*,s02}, and E a local identity operator without itags; then in order to ensure a complete local state space, one may write


```
>> getIdentityQS(A,1,'-A:~',E)
which generates a QSpace with itags {K01,s02,K02~*}.
K01 is inherited from A via 'A,1'; the remaining two itags
are derived from the remaining two itags in A;
therefore 'A,i' must have i=1 or 2 with in LRs index order.
conj-flags are properly adjusted as needed.
Conversely, getIdentityQS(A,2,'-A:~',E)
generates itags {K02,s02,K01~*}, e.g. used for R->L sweep.
```

Finally, the output index order may be changed by specifying a permutation perm to be applied on the final object. If itag is not specified, perm must be in numerical format, otherwise a (compact) string format is also accepted.

Further options

```
'-h'  display this usage and exit
'-v'  verbose
```

Note that mex files do not allow to return class objects, hence A is returned as a QSpace structure. To get a class object, use QSpace(getIdentityQS(...)), or equivalently, if getIdentity() is properly defined as a wrapper routine within matlab's Class/@QSpace (see MPS Pack), having a QSpace input A, this may be shortened to getIdentity(A,...).

(C) AW : Apr10 ; Oct14 ; May17

```
>> getQDimQS -h
```

```
Usage: [q,dd,dc] = getQDimQS(A,dim)
```

```
get dimension for every Q in QIDX for given QSpace object
dim is dimension to check (dim=='op' can be used to
consider both dimensions of an operator simultaneously)
```

Data returned

```
q    symmetry labels present (unique; n x nsymlabels)
dd   corresponding dimension of reduced data (n)
dc   corresponding dimension of Clebsch-Gordan (n x nsym)
```

AW (C) Jun 2007 ; Wb,Oct12,10

```
>> getRC -h
```

```
Usage: [...] = getRC(...)
```

```
C=getRC(cgr)  get RCStore data as specified by input  
C=getRC(<sym>,'--info');
```

```
Wb,Oct02,15 ; Wb,Nov06,16
```

```
>> getSmoothSpec -h
```

```
Usage: [omega, As [,Info]] = getSmoothSpec(om, Araw, 'eps',... [, opts])
```

```
om          omega (bins) for Araw  
Araw        raw discrete spectral data; it may contain several set  
            in separate columns, e.g. like data for different spins
```

Options / Flags

```
'om',...    new discretized set of omega values (also returned as omega)  
'sigma',... sigma used in log-Gauss broadening (0.6)  
'eps',...   smoothly replaces log-Gauss with regular Gaussian;  
            in absolute units since this routine does not know  
            anything about temperature(!). For the same reason,  
            eps must be specified (typically may choose eps=T).
```

```
'sigma2',... Gaussian width sigma2*eps for w<<eps (0.5)  
'alpha',...  shift in log-Gauss broadening  
            sigma/4: norm (=height) preserving (default)  
            sigma/2: width preserving
```

```
'func'      input data is not discrete but already function of omega  
'nlog',...  number of bins per decade of omega scale  
            for logarithmic discretization (128 = nlog_fdmNRG/2!)  
'emin',...  minimum energy for omega binning (1E-8)  
'emax',...  maximum energy for omega binning (10.)
```

```
'disp',...  verbose for disp>0 (1)
```

Note that the first (last) bin contains all data below
(above) emin (emax).

Output

```
om          omega corresponding to the energy binning [-emax,+emax]  
As          smoothened spectral function
```

see also fdmNRG.
AWb (C) May 2006

>> getSmoothTDM -h

Usage: [At [,Info]] = getSmoothTDM(tt, om, Adisc [, opts])
[At [,Info]] = getSmoothTDM(tt, ARAW, [, opts])

tt	discretized set of time dependent data
om	omega binning for Araw
Adisc	raw TDM data with dimensions [nom, nop [,nnrg]] with nom=length(om), nop=number of operators and, optionally, nnrg the number of NRG iterations
ARAW	alternatively, instead of om and Adisc, the RAW data returned by tdmNRG() in partial mode can be specified as input

Options / Flags

'disc'	discrete data (i.e. does not require d(om) weight; default)
'func'	functional data (i.e. DOES require d(om) weight) default: disc for rank(A)==3 and func for rank(A)==2
'-q'	quiet mode (reduce number of log output)
'alpha',...	exponential decay of energies $\exp(-\alpha \text{ dE})$ [0.] this applies alpha in time domain (F. Anders 2005)
'sigma',...	applies broadening in frequency domain (log.gauss) NB! either alpha or sigma can be specified but not both default: sigma=0.1 (with alpha ignored)
'Lambda',..	Lambda from Wilson discretization (required with alpha)
'eps',...	smoothly replaces log-Gauss with regular Gaussian; in absolute units since this routine does not know anything about temperature(!). For the same reason, eps must be specified (typically may choose eps=T).
'nlog',...	number of bins per decade of omega scale for logarithmic discretization (100)
'emin',...	minimum energy for omega binning (1E-8)
'emax',...	maximum energy for omega binning (10.)

Note that the first (last) bin contains all data below
(above) emin (emax).

Output

At input spectral information transformed into time
 dependent data.
Info structure with additional information

see also tdmNRG.

AW (C) Jan 2007

>> `getSymStates -h`

Usage #1: `[U,I]=getSymStates(Sp,Sz);`

get symmetry eigenstates for symmetry operations given by
raising operators Sp and (diagonal) z-operators Sz.
The order of the Sz operators determines the order of the
z-labels to be determined.

Usage #2: `[U,I]=getSymStates(SOP)`

Moreover, Sp and Sz can be grouped according to each symmetry
by using a single vector structure Is with the mandatory
fields .Sp and .Sz. In this case, an empty Sp is interpreted
as Abelian quantum number. Eventually Sp and Sz are combined
into a single sequence as in usage #1.

Further optional fields

.type symmetry type (in QSpace convention)
.qfac applied to all z- and q-labels (scalar or vector)
.jmap subsequently to qfac, applies linear transform
 of q-labels (square matrix)

Output: symmetry eigenstates written as full matrix in U,
and info structure I with the fields

basically reiterating the input

.R0 input .Sp and .Sz data
.type symmetry type
.JMap J-transformation used (identity if not specified)
.dz number of z-operators for each symmetry

actual output data

.dd dimension of irreducible multiplets generated
.RR irreducible representations with qfac and jmap applied
.QZ interleaved q- and z-labels.

.d2 dimensions of setup in QZ (non-abelian number have
 the same number appearing twice)

(C) Wb,Jul05,10

>> **isIdentityCG** *-h*

Usage: i = isIdentityCG(A)

check whether QSpace A has indentity Clebsch-Gordan spaces.
Note that for full abelian spaces, this is always true.

Wb,Jan12,12

>> **isIdentityQS** *-h*

Usage: i = isIdentityQS(A [,eps])

check whether QSpace A represents a block-diagonal
identity matrix. This also checks the corresponding
Clebsch-Gordon spaces if any (default: eps=1E-12).

(C) AW : May 2006 ; Oct 2010 ; Oct 2014

>> **makeUniqueQS** *-h*

Usage: A = makeUniqueQS(A [,opts])

MEX wrapper routine for QSpace::makeUnique()

Options

-[qQ] quite mode
-[vV] verbose mode (wrt. outer multiplicity)

AWb (C) Nov 2007 ; 2012

>> **mpsIsHConj** *-h*

Usage: [i,istr]=mpsIsHConj(A [,eps]);

check whether QSpace is hermitian within threshold eps (1E-12).
A must be even-rank object as hyperindex is allowed.

Wb,Aug17,06

>> **normQS** *-h*

Usage: nrm = normQS(A)

calculates the (Frobenius) norm of given QSpace,
i.e. $|A|^2 = \text{norm}(A)^2 = \text{trace}(A'A)$, and appropriately
generalized to arbitray rank. In the presence of CGCs, this
implies $|A|^2 = \sum_i (|data\{i\}|^2 * \text{prod}_j (|cgs(i,j)|^2))$.

(C) AW : Jun 2010 ; Oct 2014

>> **orthoQS** *-h*

Usage #1:

[A1,A2,[,info]] = orthoQS(PSI,idx, [,OPTS]);

Orthonormalize QSpace PSI w.r.t. given SINGLE index idx,
which is split off into tensor A1, connected via a trailing
new intermediate index with A2, i.e. $PSI=A1*A2$, where
A2 keeps its index order (i.e. index idx stays in place).

Note that the new intermediate index will typically be
truncated w.r.t. to singular values (see stol below).

Convention on orthonormalization direction (cf. lrdir below,
which is NOT specified here): A1 is ALWAYS an isometry/unitary
that has one in- and one out-index. This allows the new
intermediate index to carry the same itag as idx in PSI.

Usage #2 (abelian symmetries only)

[A1,A2,[,info]] = orthoQS(PSI,idx, lrdir [,OPTS]);

Similar to usage #1 above, except that for abelian symmetries,
idx may contain any number of indices consistent with PSI
(in constrast, for non-abelian symmetries, idx can only
specify a single index (or equivalently r-1 indices with
r the rank of PSI).

Usage #3 (abelian symmetries only)

[A1,A2,[,info]] = orthoQS(A1,A2,ic1,ic2, lrdir [,OPTS]);

Orthonormalize QSpace A1 towards index ic1 using SVD,
 where the resulting tensor, split off from A1, is right
 away merged with A2 at index ic2 (having lrdir='>>';
 vice versa for lrdir='<<').

This internally connects to usage #2, in that the input tensors
 A1 and A2 are right away contracted; e.g. for lrdir='>>':
 PSI = contractQS(A1,ic1,A2,ic2);

INPUT

PSI or PSI=A1*A2, where
 A[12] are two nearest neighbor tensor
 ic[12] that share the common index specified by ic1 and ic2.

lrdir direction of orthonormalization; this accepts the
 following equivalent values:
 '>>', 'LR', or +1 for "left-to-right"
 '<<', 'RL', or -1 for "right-to-left"

OPTIONS

'Nkeep',... max. number of states/multiplets to keep (-1: all)
 'Nkmin',... min. number of states/multiplets to keep on index
 connecting A1 to A2 (default: 0, i.e. no lower bound)
 'stol',... absolute tolerance on SVD i.e. on sqrt(rho)
 default: 1E-8 if Nkeep is not set, 0 otherwise.
 'itag',... itag on newly generated index
 '-v' print info

OUTPUT

A1[2] orthonormalized A1[2]
 info info structure containing (amongst other fields)
 .svd vectorized sorted complete singular value data;
 multiplicity (i.e. multiplet dimension) is specified
 in svd(:,2) in the presence of non-abelian symmetries.
 .svd2tr truncated weight (singular values squared).
 .sfac factor to reestablish the original norm(PSI).
 .flag !=0 indicating potential trouble (see .info field).
 .DD (multiplet) dimension of original PSI.
 .D1 (multiplet) dimension of A1.

See also svdQS.

AW (C) Jun 2006; Jun 2009; Aug 2015

>> **permuteQS** *-h*

Usage: A = permuteQS(A, P [, 'conj'])

permute input QSpace using given permutation P.

Optional trailing 'conj' also applies (complex) conjugation (note that this also affects real QSpaces in that qdir and itags are altered!).

For convenience, P [, 'conj'] may also be represented as single string, e.g. [2 1], 'conj' is equivalent to '2,1;*' or '21*' where the convention on string notation follows that of contraction indices [ctrIdx].

NB! [06/02/2019] the provided permutation can be shorter than the rank of the QSpace; in this case it only affects the leading range of indices, i.e., acts like an identity on the remainder of indices.

AW (C) Aug 2006 ; May 2010 ; Oct 2014

>> **plusQS** *-h*

Usage: C = plusQS(A, B [, bfac, opts])

adds one QSpace to another as in $C = A + \text{bfac} * B$ ([bfac=1])
block-diagonal identity matrix.

Options

'-v' verbose mode (e.g. on remaining outer multiplicity)

(C) AW : Jun 2006 ; Oct 2014

>> **setupRCStore** *-h*

Usage #1: setupRCStore(sym [,opts])

Setup CGC container for given symmetry,
with the data stored in directory specified by the
environmental variable \$RC_STORE.

Usage #2: setupRCStore(sym,q1,q2 [,opts])

explicitely generate tensor product decomposition $q_1 \times q_2$

Options

'npass',... number of passes through gCG data (3).
'dmax',... specify max. average multiplet dimension $\sqrt{d_1 \times d_2}$
in tensor product decomposition ($10 \times q_{len}$).
'-a' generate all, yet compatible with dmax (by default,
multiplets with 2-digit qlabels are no longer used
in decomposition)

(C) Wb,Aug07,14 ; Wb,Apr01,15

>> skipZerosQS -h

Usage: [A,n] = skipZerosQS(A [,eps,opts]);

skip data blocks that have all-zero entries to within eps
(default eps= $1E-14$). The total number of skipped blocks
is returned as 2nd argument.

For scalar (i.e. rank-2 block-diagonal) operators zero-
blocks are kept, by default (e.g. relevant for Hamiltonians
where states with energy zero are relevant states.
This can be disable by using the option '--all'.

Wb,May14,10

>> svdQS -h

Usage #1:

[U,S,Vd,[,info]] = svdQS(PSI,idx [,OPTS]);

Compute SVD decomposition of QSpace object PSI, where the
indices idx are split off into Vd, and the remaining indices
into U, which are '>>' and '<<' orthonormalized, respectively
(i.e. are unitaries or isometries generated from the SVD.
Typically, the indices in PSI are 'all-in'.

Note that the 3rd argument returns $Vd := V'$ and not V,
i.e. already includes the dagger such that $PSI = U \times S \times Vd$
[for idx=2; this is also LAPACK convention; for idx=1
the transpose is returned for U, i.e., $PSI = [(U^t) \times S \times Vd]^t$,

since, by convention, U resembles the index order of PSI].

For non-abelian symmetries, idx is required to be a single index (or conversely, all indices but one). If PSI is grouped into two sets of indices each of which has more than one index, then indices need to be fused prior to calling svdQS. Also, in this case of a single index specified, U always resembles PSI, i.e. has the same rank and index order.

Usage #2 (deprecated):

```
[U,S,V',[,info]] = svdQS(A1, A2, ic1, ic2 [,OPTS]);
```

Compute SVD decomposition of QSpace object $PSI=A1*A2$, where '*' stands for the contraction $(A1,ic1,A2,ic2)$. The split occurs between the remaining indices in A1 and the remaining indices in A2. These indices are typically 'all-in', while the index/indices ic1 to be contracted with ic2 can have any initial orientation. There will always be a single new intermediate connecting index between A1 and A2.

Input

PSI	or $PSI=A1*A2$, where
A[12]	are two nearest neighbor tensor
ic[12]	that share the common index specified by ic1 and ic2.

OPTIONS

'Nkeep',...	max. number of states/multiplets to keep (-1: all)
'Nkmin',...	min. number of states/multiplets to keep on index connecting A1 to A2 (default: 0, i.e. no lower bound)
'stol',...	absolute tolerance on SVD i.e. on $\sqrt{\rho}$ (default: 1E-8 if Nkeep is not set, 0 otherwise.)
'itag',...	itag on newly generated index
'disp'	display info to stdout

OUTPUT

A1(2)	updated and properly orthonormalized A1(2)
info	info structure containing
.SVD	singular value data blocked within QS symmetry space
.svd	vectorized singular value data

See also orthoQS.

AW (C) Sep 2008 ; Aug 2015

>> **traceQS** -h

Usage: x = traceQS(A [,i1,i2])

trace out index set i1 with i2 of given QSpace object;
for even-rank A, missing index sets is interpreted as
full trace of indices i1=1:r/2 with iw=r/2+1:r.
if all indices of A are traced out, the returned x is
a number (QSpace otherwise).

AW (C) 2006 ; 2018

MEX routines (util)

>> **filestat** -h

Usage: i=filestat(f,tag [,'-s'])

get system file information

If '-s' is specified, a time string is returned;
otherwise (by default) the returned date is in Matlab datenum
format, e.g. can be used with datestr(t,*).

Available tags

'm'	modification time stamp
'c'	creation time stamp
'a'	last access time stamp

All these time stamps return split-second resolution.

Wb,Jan28,16

>> **getchar** -h

Usage: getchar()

C++ program that reads single character from keyboard

Wb,Jan09,09

>> **kramers** -h

Usage: [yr,I]=kramers(xi,yi [,OPTS])

calculate Kramers-Kronig transform from discrete data
using linear interpolation scheme.

xi	x-data
yi	y-data
yr	kramers kronig transformed data (default: imag->real)

Options

'-disc'	yi data is already discrete
'eps',...	for continuous data: regularization of principal value data for $x_i = x_j$ ($x_i += \text{eps} * (x_i - x_{i-1}); 1E-10$)

If isdisc

'faceta',...	width of Lorentzian delta function relative to data point spacing dx (1.)
--------------	--

else

'g1flag'	include tails to infinity by assuming that input yi decays like $1/x$ for large x
'g2flag'	same as g1flag but assuming $1/x^2$ decay
'r2i'	do real->imag transform instead (=overall minus sign)

Wb,Dec12,06

>> **matchIndex** -h

Usage: [Ia,Ib,I] = matchIndex(QA, QB [,OPTS])

QA or QB are matrices whos rows are matched; consequently,
their number of columns (row dimension) must be equal.

Options

'-s'	sort indizes with respect to A
'-q'	quiet mode, i.e. do not issue warning when NaN's are encountered.
'eps',...	specify epsilon within which to accept match (0)
'-f'	use float instead of double (gets rid of numerical noise in case of double precision input data)

Output

Ia index into QA (QA.Q)
Ib index into QB (QB.Q)

I an info structure with elements
.n number of matches [=length(Ia)]
.dim number of input records [size(QA,1), size(QB,1)]
.ix1 entries in QA not matched in QB
.ix2 entries in QB not matched in QA

AW (C) 2006 ; Feb10,11

>> **mexworld** -h

This is a help on the mex-file mexworld
Wb,Jan12,19

>> **mpfr2dec** -h

Usage: [data vector]=mpfr2dec([char matrix] [,base]);

Convert mpfr data strings to decimals.
Since matlab is col-major, the data string for each
value is expected to represent an entire column.

Example

```
mpfr2dec(['0':'9', 'A':'Z', 'a':'z'],62)
```

Wb,Jul01,14

>> **printfc** -h

Usage: printfc(fmt)

C++ program that emulates C printf
ie. does not terminated automatically with a newline and
allows for escape sequences (in contrast to fprintf).

NB! If more than one onput argument is specified, all
arguments are forwarded to matlab's sprintf.

AWb (C) Jan 2006

>> **sparse2wb** *-h*

```
function S=sparse2wb(S)
```

convert matlab sparse array to wbsparse format.

AW (C) 2011

>> **uniquerows** *-h*

```
Usage: [A [,I,D]] = uniquerows(Q [,opts])
```

MEX file that makes rows of array unique. the second return argument is a cell array of index sets of rows with the same data. D specifies the corresponding multiplicity of the unique rows.

Options

'-1' return only first index in each group in I

Wb (C) Sep 2006

>> **wbeig** *-h*

```
Usage: [E,U] = wbeig(H);
```

eigenvalue decomposition based on BLAS dsyev
for (genearlized) symmetric rank-2 object.

AW (C) 2009

>> **wbhash** *-h*

```
Usage: [kk,I] = wbhash(dd,N [,opts]);
```

Compute keys kk using given hash for each row of the input array using given hash specified by the hash function (uses offset and multiplier) together with the final dimension N of the hash key (hash mod N). The hash function is given by

```
unsigned long h=offset;  
for (i=0; i<len; ++i) {
```

```

        h = mult*h + s[i];
    }

```

Here `s` represents a row in `dd` of length `len`.
 Default values for the hash function are

```

offset = 5381 (prime)
mult = 33 = 2^5 + 1;

```

which correspond to the hash function 'djb2'
 as suggested by Daniel B. Bernstein Having `mult=33`,
 the hash value is evaluated through `h = (h<<5)+h+s[i]`.

Options

For algorithm based on multiplier
 'mult',... multiplier (33)
 'offset',... offset (5381)

Alternatively the following flags use default string hashes
 '-stl' uses `hash<std::string> string_hash(char(dd))`.
 '-wbb' adapted versions of algorithms in `boost/hash.cpp`
 '-ump' test mode for `unordered_map`

Wb,Jul01,14

>> **wbhist** *-h*

Usage: `ybin=wbhist([strid,] x,y [,xbin]);`

mex routine to collect binned data. If `y` is not specified,
 this generates a simple histogram counting occurrences.

NB! The binning occurs with respect to the mid-points of
 the `xbin` data, where the first bin contains everything for
`x < mean(xbin(1:2))`, and the last bin everything for
`x >= mean(xbin(end-1:end))`.

NB! data is stored globally within mex file, i.e. is
 cleared when using 'clear functions'. However, this allows
 subsequent calls without last argument `xbin` to add to
 existing binned data. The last argument must be specified
 for the first call given some `strid`, and whenever the
 histogram data shall be initialized.

default hist set has strid 'default'.

Further tags:

'stat'	current status of wbhst (global)
'clear'	clear entire wbhst (global)
'id','remove'	remove wbhst data set associated with 'id'

Examples

```
wbhst([],[],xbin);  
wbhist(x, []);  plain count (assumes y=ones(size(x))).  
[a,b,c]=wbhist(['default']);
```

Wb, Feb12, 11

>> **wbrat** -h

Usage: [q,I] = wbrat(q [,opts]);

get rational approximation of given number
based on continued fraction approach.

Options

'-h'	display this usage
'-v'	verbose mode
'-q'	quiet mode
'-r'	assume also square root data (using rat() on q^2)
'-a'	also return continued fraction sequence in I
'nmax',...	maximum number of iterations (nmax)
'eps1',...	tolerance on continued fraction (1E-6)
'eps2',...	rel. overall tolerance wrt. to input number q (1E-12)

Output

q	same as input, but 'fixed' to rational approximation
I	info structure
	.P integer numerator
	.Q integer denominator
	.relerr relative error of rational representation
	.n total number of continued fraction iterations used
	.param (input) parameters nmax, eps1, eps2 used

See also MatLab's `rat.m`
AWb (C) July 2010

>> `wbsvd -h`

Usage: `[U,S,V] = wbsvd(H [,I]);`

singular value decomposition based on BLAS `dgesvd`
for genearlized rank-r object.
index set I specifies which dimensions are combined
into first fused dimension. For rank-2 objects
and I omitted, this corresponds to the regular SVD
on matirizes.

AW (C) Sep 2009

Class/@QSpace member functions

```
>> help QSpace/Casimir.m
```

```
function Casimir(S)
Wb,Oct05,15
```

```
>> help QSpace/QSpace.m
```

```
Function [A,varargout] = QSpace(...)
```

Construct QSpace class object.

This serves to wrap QSpace structures into QSpace objects,
and also can be used to generate several specific QSpace objects.
For historical reasons, this also includes low-level QSpace
constructors for the case of only abelian quantum numbers.

Usage:

```
#1: A = QSpace(Q,data)
    A = QSpace(A)    conversion e.g. from struct
    A = QSpace(A,'check') also performs full check on QSpace structure
```

```
#2: A = QSpace(q1, data1, q2, data2, ...) % see initQSpace
```

```
#3: A = initQSpace(Q1,M,DB)
    square operator M with row/col Q1 block size given by DB
```

```
#4: A = initQSpace(Q1,Q2,M,'map')
    matrix M with row/col QIDX given by Q1/Q2
    eg. used for projectors
```

```
#5: A = QSpace(Q,M,'operator');
```

```
#6: A = QSpace(A,'unity');
```

```
#7: A0=initQSpace(QL,Qs1,Qs2,...); setup identity tensor
    for product spaces (QL,Qs1,...)
```

it must hold: $\text{size}(QL,2) = \text{size}(Qs1,2) = \text{size}(Qs2,2) = \dots = \text{QDIM}$.
Qs and QL span the full Hilbert space, hence certain quantum
lables may appear several times.

Wb,May12,06 ; Wb,Apr16,16

>> **help QSpace/SEntropy.m**

```
function [se,rr,I]=SEntropy(Rho [,eps,opts])
```

Calculate von Neumann entropy of the mixed (reduced)
density matrix R.

R may also contain non-abelian symmetries in which case
the output weights rr are already weighted by their
corresponding multiplet degeneracies such that sum(rr)=1.

Options

eps check trace(Rho)=1 within numerical accuracy of eps (1E-12)
'-xm' expand multiplet degeneracy in returned rr
'beta',... assume R is a Hamiltonian => apply beta=1/T and exponentiate

Wb,Jan24,11

>> **help QSpace/SetCC.m**

```
Function [A,t]=SetCC(A [,idx])
```

Set "dagger" for specified index-labels (all if idx is not specified).
If any A.data{k} is complex, A.data will also be complex conjugated.

Wb,Aug04,12

>> **help QSpace/SetOL.m**

```
function A=SetOL(A,X,k [,otag])    % usage #1  
function A=SetOL(A,itag[,otag])    % usage #2
```

Set operator labels of the type { itag, itag* [, 'op*'] }.
default operator tag (otag) for rank-3 tensors is 'op'.

Usage #1

set index labels for operator A based on the label found
in tensor X at dimension k.

Wb,Aug04,12

>> help QSpace/acomm.m

```
function C=acomm(A [,B,cflag])
```

cflag if set, must equal 'conj'
and is interpreted as conjA, i.e. $C = \{A', B\}$

Wb, Feb06, 16

>> help QSpace/appendScalarSymmetry.m

```
function A=appendScalarSymmetry(A,qtype [,opts])
```

Append scalar symmetry (e.g. new symmetry label,
where all existing states get a newly added qlabel q=0
with the zeros repeated according to the rank of the
added symmetry.

Options

'pos',... actually insert symmetry at particular position
in symmetry order; e.g., 'pos',1 will prepend the symmetry.
'q',... specify particular symmetry labels other than (default q=0).
if non-zero, this assumes A to be of operator type in that
specified q is applied to the first two indices only *)

*) rather than specifying non-zero q, consider starting with zeros
first, followed by mapping of operators into tensor product
of state spaces of operators

See also:

Wb, Jul12, 11 ; Wb, Dec08, 14

>> help QSpace/appendSingletons.m

```
function A=appendSingletons(A,'new_itag[*]')
```

```
function A=appendSingletons(A,qdir)
```

```
function A=appendSingletons(A,r_new,qdir)
```

Add singleton dimension(s) to reach rank r_new.
qdir sets direction of new singleton dimensions (numeric; e.g. +/-1)

In usage #1 assigns the specified itag to the newly added singleton
dimension; this also permits to simultaneously specify qdir via a

trailing conjugation flag '*'.

If `numel(qdir)>1`, it is assumed it specifies `qdir` for all of `A` where the values in `qdir` are ignored for the ones already existing in `A` [this is useful if `qdir` was obtained via `getqdir(Aother)`].

NB! use `makeIrop()` to append `irop` dimension instead of this, since there may be non-zero symmetry labels for all-abelians ops.

Examples

```
A4=appendSingletons(E,' +-')
```

See also `makeIrop`, `fixScalarOp`.
Wb,Sep17,18

>> **help QSpace/applyK1K2.m**

```
function [S1,S2]=applyK1K2(S,H)
```

auxilliary routine that applies energy kernels
K1 (principal value) and K2 (remainder for diagonal values).
e.g. required for static magnetic susceptibility
(K2 accounts for correction due to degeneracies).

Wb,Jul05,12

>> **help QSpace/applyZ.m**

```
function A=applyZ(A,idx [,iq,opts])
```

apply fermionic signs to given index / indices (`idx`)
with fermionic charge (or parity `Z2`) assumed at position
`iq` in the `Q`-labels (`iq` may be skipped, if only one
abelian symmetry label is present).

Options

'qfac',... : factor to apply on charge labels, to obtain
actual charge (must be integer; enforced); e.g. note that
single-flavor spinless fermions have an artificial factor
of 2 on their charge labels to permit simple symmetry around
half-filling; hence in this case: `applyZ(...,'qfac',1/2);`

Wb,Sep17,18

>> **help QSpace/blkdiag.m**

```
function M=blkdiag(A,varargin)
```

Wb,Aug10,16

>> **help QSpace/cat.m**

```
function X=cat(A1,A2,...,k)
```

Concatenate data sectors of given QSpaces along dimension k
(QSpace vectors are also included in the concatenation procedure).
NB! k may be a cell, i.e. {k,qdir} to specify qdir of newly added
dimensions.

Wb,Sep17,18

>> **help QSpace/cgr_size.m**

```
function s=cgr_size(A,i,j)
```

Wb,Jan08,12

>> **help QSpace/checkOM.m**

```
function m=checkOM(A [,opts])
```

Check for outer multiplicity in given QSpace

Wb,Feb15,14

>> **help QSpace/checkSameSpace.m**

```
function i=checkSameSpace(A,ia,B,ib)
```

check whether QSpaces A and B share the same vector spaces
(simple check in terms of symmetry blocks and respective
multiplet [and Clebsch-Gordan dimensions, if applicable]).

This routine uses getQDimQS(A,ia) [and similar for B]
hence ia accepts the same index specification as getQDimQS
that is, a specific dimension or 'op'(erator).

Wb,Jan12,12

```
>> help QSpace/checkdim.m
```

QSpace/checkdim is a function.

```
[ok, ii] = checkdim(A, pp)
```

```
>> help QSpace/chopd.m
```

```
function A=chopd(A [,aref])
```

apply chopd() to data

=> sets small matrix elements to zero

=> removes imaginary noise if any

Wb,Oct17,11

```
>> help QSpace/comm.m
```

```
function C=comm(A [,B,'conj','+'])
```

Compute commutator (or anticommutator, if '+' is specified).

if B is omitted, this assumes B=A && 'conj'.

'conj' if set, is interpreted as conjA, i.e. $C = \{A', B\}$

For anticommutator, use comm(A [, B, cflag], '+');

The operators and B must be rank-2 or rank-3, where the latter case assumes operator index order (ss',o), i.e. the irop (=spinor) index comes last.

Ex. comm(F,'+') computes $\{F', F\}$

Wb,Feb06,16

```
>> help QSpace/compact.m
```

merge quantum numbers of tensor operator

Wb,Jul07,09

```
>> help QSpace/conj.m
```

Function: A=conj(A)

get complex conjugate of operator in QSpace format;
overloading the conj routine

Wb,Jun02,08

>> help QSpace/contract.m

```
function A=contract([args])
```

same as contractQS

but returns QSpace object (rather than plain structure).

Wb,Sep18,06 ; Wb,Mar31,16

>> help QSpace/ctranspose.m

overloading the hermitian conjugate operator '

accept hyper index

Wb,Nov30,09

>> help QSpace/datasize.m

```
Function [D,DD]=datasize(A [, '-s'])
```

get full listing of size(data{ }),

except if option '-s' is specified which gives summary

in terms of total size in bytes of A.data

Wb,Apr24,08

>> help QSpace/decomposeOp.m

```
function [cc,err,I]=decomposeOp(H,{01,02,...}[,R][,opts])
```

decompose operator H into operator (sets) specified in X={01, 02, ...};

if R is specified, operator overerlap is calculated

w.r.t. density matrix R.

Wb,Apr16,13

>> help QSpace/deconj.m

```
Function [A,t]=deconj(A [,idx])
```

Unset "bra" (complex conjugate) label for specified dimensions

(all if idx is not specified).

Wb,Apr17,13

>> help QSpace/diag.m

Function: A=diag(A [,OPTS])

extract diagonal of operator in QSpace format
alternatively, if only diagonal is stored,
diagonals become full again.

Options

Wb,Sep08,06

>> help QSpace/dim.m

function s=dim(A [,opts])

Options

'-a' all (simply return output from getDimQS())
'-f' full dimension
(in case of CGC spaces, corresponds to state space dimension)
'-op' IRQP dimension (i.e. checking for rank-3 QSpace)

Wb,Jul31,12

>> help QSpace/disp_cgr.m

function disp_cgr(A)

see also plot_cgr()

Wb,Jan13,12

>> help QSpace/disp_qdim.m

function disp_qdim(A,varargin)

example: disp_qdim(A0,2,IS.SOP)

Wb,Jan26,12

>> help QSpace/display.m

Function display(A [,OPTS])

Options

'-a' show all entries

```
'-v'      verbose flag (shows CGS dimension separately for each symmetry)
'-E'      sort QSpace wrt. data (energy) [assumes diagonal Hamiltonian]
'-s'      sort QIDX
'sperm',... sort QIDX using given permutation
```

By default, QSpaces with many entries (length > 14) will show a truncated list, including also the two largest entries.

Wb,Mar01,08

>> help QSpace/dsize.m

```
Function [D,[DD|I]]=dsize(A [,opts])
get maximum D for every dimension of given MPS state
```

where D is a vector of length rank(A)

Options

```
'-d' returns detailed block sizes of single input A in DD with
CG space considered if exists (overall size returned in D)
```

Wb,Aug08,06

>> help QSpace/eig.m

```
function [varargout]=eig(H,varargin)
    ee=eig(H,varargin);
    [A,E]=eig(H,varargin);
```

Wb,Aug30,12

>> help QSpace/eq.m

overload of == (equal) operator

>> help QSpace/expandQ.m

```
Function qq=expandQ(A,k)
deprecated: see getQRange.m
```

```
>> help QSpace/finditag.m
```

```
function i=finditag(A,pat [,n])
```

```
    find itag with specit regexp pattern
```

```
    if n is specified, the result must yield exactly n matches.
```

```
Wb,Jun03,19
```

```
>> help QSpace/fixAbelian.m
```

```
function fixAbelian(A,B,...)
```

```
Options
```

```
    -xop3    reduce "scalar" rank-3 operators to regular rank-2.
```

```
    -addCGR  add (trivial) CGR space to all-abelian operators.
```

```
    -op3     together with -addCGR: make operator of rank 3 if rank 2.
```

```
Wb,Feb20,13
```

```
>> help QSpace/fixScalarOp.m
```

```
function fixScalarOp([opts,] A,B,...)
```

```
    reduces "scalar" rank-3 operators to regular rank-2.
```

```
    converting (yet keeping) the Clebsch Gordan coefficient  
    spaces to sparse.
```

```
Options
```

```
    '-f'    enforce reduciblity if rank-3.
```

```
See also makeIrop() to append 3rd dimension
```

```
Wb,Aug21,13
```

```
>> help QSpace/full.m
```

```
wrapper routine for mpsFullQS()
```

```
>> help QSpace/full2.m
```

```
wrapper routine for mpsFull2QS()
```

```
>> help QSpace/gdisp.m
```

```
function gdisp(A)
```

graphical information regarding given QSpace A.

Wb, Feb09, 11

```
>> help QSpace/getAtensorLoc.m
```

```
function A=getAtensorLoc(X [,qvac][,l])
```

get Atensor by extracting local state space from local operator X together with vacuum state on virtual bond (default for qvac; optionally any other bond-space can be specified as third argument, instead).

l \in {'L','R'} specifies the location of vacuum state using LR's index order (default: l='L').

All arguments except for the last (l) are directly handed over to getIdentityQS(); hence this also allows more general QSpace tensors X and qvac together with the explicit specification of the space index to use.

Wb, Dec28, 14

```
>> help QSpace/getData.m
```

QSpace/getData is a function.

```
B = getData(A)
```

```
>> help QSpace/getEQdata.m
```

```
function [E,Q,D]=getEQdata(HK [,opts])
```

Wb, Sep22, 12

```
>> help QSpace/getId.m
```

```
function A=getId(A,qs [,k])
```

reduce QSpace A to rank-2 Id tensor which only has vacuum state / scalar representation with each index.

if k is specified, itag{k} will be used.

Wb,Oct18,14

>> help QSpace/getIdentity.m

```
function A=getIdentity([args])
```

same as getIdentityQS
but returns QSpace object (rather than plain structure).

Wb,May28,16

>> help QSpace/getIdentity3.m

```
function E=getIdentity3(A,idx [,tag,opts])
```

Map pair of open indices to be combined as in δ_{ij}
into single index in $E_{(i,j,k)}$ by mapping $(i=j \rightarrow k)$.
The added third index therefore always transforms like a scalar
for all symmetry sectors present, i.e., has q-labels all zero.

Options

--rho normalize E
such that it resembles density matrix in (i,j) for fixed k.
tag tag assigned to the new trailing index.

Wb,Aug28,20

>> help QSpace/getQRange.m

```
Function qq=getQRange(A,k)
```

>> help QSpace/getQSpectra.m

```
function [q,EE,qq,dz,DZ]=getQSpectra(H,ws)
```

get symmetry-resolved energy spectra.

Input

H diagonal scalar operator (with diagonals stored only)

ws specifies which symmetry to pick

Output

q unique set of labels for symmetry ws
EE cell array of energies, one cell for each symmetry in q
qq full set of symmetry labels (expanded to same size as EE)
dz full set of multiplet degeneracies
DZ full set of multiplet degeneracies (expanded to same size as EE)

Wb,Apr12,12

>> help QSpace/get_cgr_fac.m

```
function [dfac,s]=get_cgr_fac(A,i)
```

CGData is no longer stored with the QSpaces themselves,
only their weights cgw within outer multiplicity space.
Therefore CGData is globally normalized to 1. up to
outer multiplicity.

Options

sflag=='s' => also return range as string info

Wb,Oct17,14

>> help QSpace/get_qdim.m

```
function [dd,D]=get_qdim(A,dim)
```

get cgr dimension for given dimension

Wb,Nov27,15

>> help QSpace/getitags.m

```
function t=getitags(A)
```

return itags as cell array.

adapted from QSpace/gotITags.m

Wb,Apr10,14

>> help QSpace/getproj.m

Function P=getproj(H [,eps,opts])

get projector with respect to H into ground state space
(diagonalize if necessary); eps determines the window wrt.
ground state energy, within which state are kept (1E-8).

Options

'-q' quiet mode

Wb,Aug30,12

>> help QSpace/getqdim.m

function [dd,ss]=getqdim(A)

get symmetry specific length of q-labels

Wb,Sep18,11

>> help QSpace/getqdir.m

function t=getqdir(A [,'-s'])

qdir in numeric format (+- => [+1,-1])
NB! input arguments are directly handed over to getitags.m

Options

'-s' return qdir as string (default: numeric; see above)

Wb,Mar30,15

>> help QSpace/getqfmt.m

function f=getqfmt(A [,opts])

get format string for a single $Q_{i}(j,:)$

Wb,Dec14,15

```
>> help QSpace/getscalar.m
```

```
Function a=getscalar(A)
Wb,Aug28,08
```

```
>> help QSpace/getsub.m
```

```
Usage #1: function [A,B]=getsub(A,idx)
```

Select given subspace where idx represents direct index into records of QSpace.

```
Usage #2: function [A,B]=getsub(A,q,dim)
function [A,B]=getsub(A,{q} [,dim]) // Wb,Feb21,21
```

Select subspace that has given symmetry sector(s) in given dimension(s). if q is a cell array, then this directly relates to dimensions in A e.g. getsub(ops,{[],[],0}) = getsub(ops,0,3) picks scalar sector on 3rd dimension; here '0' or 0 can be used as a shortcut to zeros(1,m) in q, with m the number of symmetry labels.

```
Usage #3: function [A,B]=getsub(A,'-0') // Wb,Dec04,20
function [A,B]=getsub(A,'-0!')
```

Assuming A is an irop (rank-3 with irop-index on 3rd leg) select scalar operator part (q-labels 0); this is equivalent to usage #2, having getsub(A,[0 0 ..],3));

in case of '-0!', also remove singleton irop dimension (in case it is not a singleton, i.e., by having multiple q=0 multiplets, an error is issued)

Options

```
-d return A.data (rather than A itself)
```

If second output argument is requested, this will contain whatever was left in A_in as compared to what was selected in A_out.

Wb,Aug20,08 ; Wb,Jun19,13

```
>> help QSpace/getsym.m
```

```
function [ss,s]=getsym(A [,dim,opts])
```


Options

- c return symmetries split as cell array
- d get dimensions (rank) of each symmetry [returns 1 for Abelian]
- r get symmetry-rank for each symmetry [returns 0 for Abelian]
- 'ir',dim index range in q-labels for given dimension
- '-I',dim get extended info structure for given symmetry

Wb,Oct24,11

>> help QSpace/getvac.m

```
function A=getvac(A [,k])
```

reduce QSpace A to rank-2 Id tensor which only has vacuum state / scalar representation with each index. if k is specified, itag{k} will be used.

Options

- 1d reduce to rank-1 vector (NB! only permitted for scalar representation with all q-labels zero!)

Wb,Oct18,14

>> help QSpace/getzdim.m

```
function [dd,qq]=getzdim(A [,dim,opts])
```

get multiplet dimensions of QSpace A.

Options

- p product of multiplet dimensions
- x expand to match data (repmat)
- u get dimension of unique qlabel set (returned as 2nd output argument)

The specification of the dimension (dim) may also take the following string values:

- '-op' gets dimension of IROP set
- 'op' gets dimension of state space dimension for scalar operator (together with '-u' this is equivalent to 'op') // deprecated

Wb,Jan24,11

>> help QSpace/gotCGS.m

```
function i = gotCGS(A)
Wb,May16,10
```

>> help QSpace/gotITags.m

```
function r = gotITags(A)

    return number of itags (if any).
    second return argument contains listing of ilables as cell array.
```

Wb,Aug04,12

>> help QSpace/got_orthogonal_cgw.m

```
function got_orthogonal_cgw(A)

    (double) check whether A got orthogonal cgw spaces
```

Wb,Feb19,16

>> help QSpace/hasQOverlap.m

```
Function: [ss,qq] = hasQOverlap(A,i1,B,i2)
check overlap in QIDX
Wb,Jun22,07
```

>> help QSpace/imag.m

```
Function: A=imag(A)

    extract imaginary data of operator in QSpace format
```

Wb,Oct19,11

```
>> help QSpace/info.m
```

```
function info(A [,istr,cflag])
```

```
    header routine used in display()
```

```
    Options: istr (info string) and cflag (compact info).
```

```
Wb,Mar01,08
```

```
>> help QSpace/info2.m
```

```
function info2(A,[dim,opts])
```

```
Options
```

```
    dim  if dimension is specified, plot will be generated
```

```
    -ll  log-log scale on plot
```

```
show further size info // Wb,Sep17,11
```

```
>> help QSpace/info3.m
```

```
function info3(A,dim [,opts])
```

```
graphical analysis of size distribution of CGC spaces
```

```
>> help QSpace/isAbelian.m
```

```
function i=isAbelian(A [,opts])
```

```
Wb,Feb20,13
```

```
>> help QSpace/isOpConj.m
```

```
function i=isOpConj(A [,output_type])
```

```
Wb,Jun06,19
```

```
>> help QSpace/isOrtho.m
```

```
function a=isOrtho(A)
```

```
Wb,Jan15,15
```

```
>> help QSpace/isQSpace.m
```

```
function [i,s]=isQSpace(A [,opts])
```

```
    overloaded routine (see also MEX/isQSpace.m)
```

```
see mpsIsQSpace.m for more information.
```

```
Wb, Feb15,13
```

```
>> help QSpace/isdiag.m
```

```
function [isd,estr]=isdiag(A)
```

```
    check whether QSpace is in diagonal rank-2 format.
```

```
Return value isd:
```

- 1 if block-diagonal in symmetry space
- 2 if, in addition, all data{} sets are row-vectors, e.g.
 compressed format as returned bei QSpace::EigenSymmetric()
- 3 same as 2, but if all entries in data{} are column-vectors
- 0 otherwise

```
Options
```

```
'-d'  If all data blocks are scalars, i.e., of dimension 1x1,  
      by default, this returns isd=1; with '-d' this returns  
      isd=2, instead.
```

```
Wb, Apr24,09
```

```
>> help QSpace/isempty.m
```

```
function ie=isempty(A)
```

```
Wb, May12,06;  Wb, Aug23,08
```

```
>> help QSpace/isequal.m
```

```
function i=isequal(A,B [,opts])
```

```
    overloading isequal and ==
```

```
    where input A or B can also be struct( QSpace object ).
```

```
    Note that the usage A==B is equivalent to isequal(A,B),
```

i.e. this assumes the default setting without any options.

Options

'-l' lenient (ignore difference in itags)
'-f' enforce all fields to be the same (no lenience at all)

Wb,Nov09,09 ; Wb,Jun28,21

>> help QSpace/isfield.m

wrapper for builtin isfield
Wb,Feb26,16

>> help QSpace/isop.m

function is=isop(A)
Wb,Nov25,11

>> help QSpace/isreal.m

function i=isreal(A)

checks whether A contains complex data
(this overloads matlabs isreal() function).

Wb,Apr23,15

>> help QSpace/isscalar.m

function i=isscalar(A [,opts])

Checks whether QSpace is a scalar, meaning rank-0
with a single number in data [e.g., a scalar is obtained
by tracing/contracting all indices resulting in a single
number].

Options

-l lenient (also consider a QSpace with non-empty Q
a scalar, as long as data contains a single number)
-x extended mode (in case of input QSpace array,
show isscalar() for each individual QSpace

Wb,Aug04,08

>> **help QSpace/isscalarop.m**

```
function [s,dd]=isscalarop(A [,opts])
```

requirements for scalar operators:

- (1) rank-2
- (2) block-diagonal in symmetries
- (3) all cgc spaces are identities

Options

-1 lenient (also accept rank-3 operators)

See also QSpace/issingleop for non-scalar, yet still rank-2 operators.
Wb,Jul05,12

>> **help QSpace/issingleop.m**

```
function [s]=issingleop(A [,opts])
```

check for possibly non-scalar, yet still rank-2 operators.

See also QSpace/isscalarop for scalar, i.e. block-diagonal
(rank-2) operators.

Wb,May11,13

>> **help QSpace/issingleton.m**

```
Function i=issingleton(A,kk)
```

Wb,Apr24,08

>> **help QSpace/itag2iter.m**

```
function k=itag2iter(A)
```

extract site position to k from A.info.itags
with expected itag format '[non-number]##[non-number]'

Wb,Aug01,17

```
>> help QSpace/itagrep.m
```

```
function A=itagrep(A [,idx], )
```

```
    Apply given regexprep() pattern to specified itags  
    idx specified a subset of itags to consider;  
    (otherwise, by default, all itags are considered)
```

```
Wb,Jan14,15
```

```
>> help QSpace/itags2int.m
```

```
function k=itags2int(A)
```

```
    Extract integer indices carried with itags,  
    assuming the itag format '[non-number]##[non-number]',  
    where outgoing indices (i.e., which carry a trailing  
    conjugate flag '*') are returned as negative integers.  
    If no valid number is encountered, nan is returned  
    for that index.
```

```
Wb,Nov19,21
```

```
>> help QSpace/itags2odir.m
```

```
function odir=itags2odir(A)
```

```
Wb,May15,17
```

```
>> help QSpace/join.m
```

```
function A=join(A,idx)
```

```
Wb,May19,10
```

```
>> help QSpace/kron.m
```

```
overloading kron()
```

>> help QSpace/logical.m

```
function q=logical(A)
```

Overload of conversion to logicals (unary operator)

e.g. for conditionals: if A, ...; else ...; end

see also QSpace/ [which returns ~logical()]

Wb,Feb18,21

>> help QSpace/makeIrop.m

```
function A=makeIrop(A [,t_op])
```

make A an irreducible operator with a proper 3rd irop index,
which thus is only intended for rank-2 scalar operators.

Operators already in rank-3 format are checked for the
right format.

Options

t_op specifies itag for the irop leg, only if added
(empty by default).

See also: fixScalarOp, appendSingletons.

Wb,Feb12,16

>> help QSpace/mapQ.m

QM = Q map that transforms Q numbers to 'rotated' set

Example: SIAM: (N1,N2) --> (Q,Sz)

$$QM \quad * \quad Q_old \quad + \text{offset} \quad = \quad Q_new$$
$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} * \begin{bmatrix} N1 \\ N2 \end{bmatrix} + \begin{bmatrix} -1 \\ 0 \end{bmatrix} = \begin{bmatrix} Q \\ Sz \end{bmatrix}$$

Wb,Aug10,6

>> help QSpace/minus.m

overloading the + operator


```
>> help QSpace/mrdivide.m

overloading * operator
Wb,Sep28,12

>> help QSpace/mtimes.m

function C=mtimes(A,B,ia,ib)
overloading * operator (matrix multiply)

>> help QSpace/norm.m

wrapper routine for normQS()

>> help QSpace/norm2.m

wrapper routine norm2 -> normQS()
Wb,Oct18,14

>> help QSpace/not.m

function q=not(A)

    Overload of conversion to logicals (unary operator)
    e.g. for conditionals: if ~A, ...; else ...; end

Wb,Feb18,21

>> help QSpace/numsym.m

function ns=numsym(A)

    determine number of symmetries in QSpace(s) A.

Wb,Feb06,14

>> help QSpace/olap.m

function x=olap(A [,B,opts])

    calculate frobenius norm / overlap.

Wb,May27,11
```

```
>> help QSpace/oplus.m
```

```
function C=oplus(A,B [r12,opts])
```

Direct sum (latex oplus) of two tensors A and B along dimension r.
If r12 is not specified, an MPS or MPO like setting is assumed
with L/R as first and second index, i.e., r12=[1 2].

Options

'first'	direct sum only in index r12(2)
'last'	direct sum only in index r12(1)
	otherwise: direct sum in both indices r12
'bfac',...	apply bfac to B prior to extending A

E.g. see <https://mathworld.wolfram.com/MatrixDirectSum.html>
Wb,Nov23,20

```
>> help QSpace/permute.m
```

Usage: A=permute(A)
permute QSpace with given permutation
Wb,Sep08,06 ; Wb,Nov24,14

```
>> help QSpace/plotEdata.m
```

```
function plotEdata(H [,opts])
```

plot eigenspectrum of given QSpace,
expecting scalar operator upon input!

see also QSpace/
Wb,Feb15,13

```
>> help QSpace/plotQSpectra.m
```

```
function Iq=plotQSpectra([H|R] [,opts][,SOP])
```

graphical analysis of eigenspectrum of Hamiltonian
(or entanglement spectrum of density matrix; requires '-ES' flag)
separating the data into its symmetry subspaces.

Options

'-ES' entanglement spectrum (expects Rho as input)
 '-x' show state space dimensions (rather than multiplet dimensions)

'yl',... ylim range
 'y2',... y-range within which degeneracies are indicated
 'ws',... which symmetries to pick (e.g. [1 2] for the first two)

SOP more verbose description of symmetry labels, which also
 appears in the title of the respective panels.
 SOP may simply be specified through IS.SOP, i.e. as returned
 by getLocalSpace() in the NRG setup; all that is used, however,
 are the strings SOP(:).info, hence, alternatively, SOP may
 only contain {SOP.info}. keywords looked for are 'spin' or
 'charge' which effects the xlabel.

Adapted from getrhoESpectra.m
 Wb, Apr11,12

>> **help QSpace/plot_op.m**

function plot_op(S,H)

 S operator (possibly IROP)
 H Hamiltonian (in diagonal representation) => show energy range

Wb, Aug20,12

>> **help QSpace/plus.m**

overloading the + operator

>> **help QSpace/randomize.m**

function A=randomize(A [,nrm,opts])

 Randomize A.data using randn()
 while preserving overall norm (unless explicitly specified).

Options

 -z use complex random numbers (default: real)

Wb, Nov11,09

```
>> help QSpace/rank.m
```

```
overloading rank function
```

```
>> help QSpace/real.m
```

```
Function: A=real(A)
```

```
extract real data of operator in QSpace format
```

```
Wb,Oct19,11
```

```
>> help QSpace/rgbQSpectra.m
```

```
function [cc,Iout]=rgbQSpectra(H,q3 [,opts])
```

```
Encode given energy spectrum to rgb spectrum  
by using thermal weights in specified (or otherwise dominant)  
symmetry sectors
```

```
Options
```

```
'beta',... effective inverse 'temperature' to use to compute  
           'thermal' weight in symmetry sectors (2)  
'-R'      assume input is density matrix which already gives  
           weights (with beta ignored here)
```

```
Wb,Sep09,21
```

```
>> help QSpace/rmitags.m
```

```
function A=rmitags(A)
```

```
skip itags of given QSpace(s)
```

```
*** NB! deprecated; rather use untag ***
```

```
Wb,Apr14,16
```

```
>> help QSpace/rmtrace.m
```

```
function A=rmtrace(A)
```

```
remove trace from input tensor A
```

```
Wb,Apr24,13
```

>> help QSpace/sameas.m

```
function i=sameas(A,B [,eps=1E-10])

    check whether QSpaces A and B are the same
    except for minor differences such as
    info.cgr.cid(end) and info.cgr.nnz
```

Wb,Aug08,16

>> help QSpace/setitags.m

```
function A=setitags(A,...)

    set itag(s) while preserving conj flag.

Usage #1: A=setitags(A,{t1,t2,...} [, [i1,i2,...]])

    set specified or all itags
    setitags(A,{'K1','K2','s2'})      sets itags { 'K1','K2','s2' }
    setitags(A,{'K','K','s'},{1,2,2}) same (numbers are appended to itags!)
```

Usage #2: A=setitags(A,tt,i)

```
    set specific itags tt to itag{i}
```

Usage #3: set itags for A-tensor (conj-flags always remain in place)

```
A=setitags(A,'-A',k)          {'K<k-1>','K<k>','s<k>'}
A=setitags(A,'-A:K,K,s', k)   {'K<k-1>','K<k>','s<k>'} more specific
A=setitags(A,'-A>K,K,s@n',k)  same, also using n-digit format (default: 2)
A=setitags(A,'-A<K,K,s@n',k)  {'K<k>','K<k+1>','s<k>'} i.e. R->L order
```

Usage #4: set itags for operator

```
S=setitags(S,'-op:X', k )      {'<X><k>','<X><k>','op'}
S=setitags(S,'-op:X','s')      {'<X>','<X>','s'}
S=setitags(S,'-op:<regex>', A)  set itags based on behavior
                                that mimicks contractQS
```

Usage #5

```
A=setitags(A,ia,B,ib);  copy itags from B to A
```

Usage #6: any of the usages above, but return itags rather than A,
by requesting 2 output args, e.g.

```
[tt,i]=setitags(A,...)
[tt,~]=setitags(A,...)
```

this returns itags+indizes themselves, rather than setting the itags in A (the conjugation is inherited from A; hence only a single QSpace A is expected).

Wb,Mar24,16 ; Wb,May28,18

>> **help QSpace/showdeg.m**

```
function [I]=showdeg(H [,opts])
```

show degeneracy of given rank-2 operator, either in diagonal representation, or to be diagonalized.

Options

```
'dE',...  considere values degenerate within dE (1E-3)
'ng',...  number of degenerate groups to display
'-f'      force flag
'-v'      always print lowest part of spectrum
'-V'      more verbose still.
```

Wb,Apr12,11

>> **help QSpace/skiprecs.m**

```
function A=skiprecs(A,ix)
Wb,Aug01,12
```

>> **help QSpace/skipzeros.m**

```
function A=skipzeros(A [,eps,'--all'])
Wb,Nov10,07
```

>> **help QSpace/sort.m**

```
function [A,isd]=sort(A,perm)
```

sort records in QSpace according to q-labels after using permutation perm (default: Id). Alternatively, perm may

also be specified as

'-E' (sort by energy) or
'-R' (same as '-E', but sorts in descending order).
'-S' sort wrt. to size of data+cgr

Wb,Sep12,06

>> help QSpace/split_ops.m

```
function [A,nsp]=split_ops(A [,opts])
```

split composite operators back to (irreducible) operator sets
that have well defined irop symmetry labels
(returns A -> A(:) expanded along dim2, such that
A(k) still references a (partial) original QSpace).

Options

--set-op set as operator type
--s2f move scalar operators to front
--trans return transpose array
--vec return as serialized vector
--cell return as cell array

Wb,Aug31,19

>> help QSpace/sqrt.m

```
function B=sqrt(A)
```

Compute sqrt(A) for diagonal QSpace(s) A
(error is issued if A is not diagonal)

Wb,Mar11,19

>> help QSpace/squeeze.m

```
Function [A,I]=squeeze(A [,OPTS])
```

remove singleton dimensions (similar to matlab's routine)

Options

```
'-start'  only singletons at the front  
'-end'    only singletons at the end (default: all)
```

Wb, Apr24, 08 ; Wb, Dec01, 20

>> **help QSpace/subsasgn.m**

subsasgn

Reference <http://www.cs.ubc.ca/~murphyk/Software/matlabObjects.html>
Classes and objects in matlab: the quick and dirty way
Kevin Murphy, 2005

>> **help QSpace/subsref.m**

subsref

Reference <http://www.cs.ubc.ca/~murphyk/Software/matlabObjects.html>
Classes and objects in matlab: the quick and dirty way
Kevin Murphy, 2005

>> **help QSpace/sum.m**

overloading the sum operator

>> **help QSpace/symperm.m**

function A=symperm(A,perm)

Permute symmetries according to specified permutation.

Wb, Feb06, 14

>> **help QSpace/symrank.m**

function r=symrank(A)

Wb, Jan09, 17


```
>> help QSpace/tensor.m
```

```
function C=tensor(A,B,...)
kron including merging of quantum numbers
Wb,Jul07,09
```

```
>> help QSpace/times.m
```

```
overloading .* operator
Options
```

```
--skip-cgc  C inherits CGC space of tensor A *as it is*
             (yet only for the symmetry sectors shared with B).
```

```
Wb,Jul04,12
```

```
>> help QSpace/toStr.m
```

```
QSpace/toStr is a function.
  qs = toStr(A, varargin)
```

```
>> help QSpace/trace.m
```

```
Function x=trace(A,I [,opts])
```

```
trace of QSpace operator
```

```
Usage 1: A=trace(A)   - regular trace of rank-2 object
Usage 2: A=trace(A,I)
```

```
generalized trace with pairwise specification of indices
to contract I = [i1 i2; j1 j2; ... ]
```

```
Wb,Sep11,06 ; Wb,Apr10,15
```

```
>> help QSpace/transpose.m
```

```
overloading the .' operator (element wise transpose)
```

```
>> help QSpace/tstfun.m
```

```
function tstfun(A)
Wb,Feb23,11
```

```
>> help QSpace/uminus.m
```

```
function A=uminus(A)
```

```
    overloading the unary minus operator (as in -A)
```

```
Wb,Sep17,06
```

```
>> help QSpace/untag.m
```

```
Usage #1: function A=untag(A)
```

```
Usage #2: function untag(A1,A2,...)
```

```
    remove itags from specified A's;
```

```
    if no return argument is requested, the modified A's  
    are assigned back to caller by their inputname.
```

```
Wb,May11,17
```

```
>> help QSpace/uplus.m
```

```
function A=uplus(A)
```

```
    overloading the unary plus operator (as in +A)
```

```
Wb,Sep17,06
```